

DATA ENGINEERING CONCEPTS

- **ETL**: extract, transform, then load. **ELT**: load raw data first, transform inside the warehouse (common with cloud warehouses); keeps raw data for reprocessing.

Store	What	Use
Data lake	raw files in cheap object storage (S3, GCS)	all data, ML, schema-on-read
Data warehouse	modelled, columnar tables (BigQuery, Snowflake, Redshift)	BI, SQL analytics, schema-on-write
Lakehouse	lake + warehouse features (Delta, Iceberg, Hudi)	both

- **Batch** (nightly/hourly) vs **streaming** (events as they arrive: Kafka, Kinesis, Flink, Spark Structured Streaming). Choose batch unless latency truly matters.
- **Orchestration**: Airflow/Dagster/Prefect schedule tasks as a DAG with retries and alerts. **dbt** for SQL transformations with tests.
- **Good pipelines: idempotent** (re-running gives the same result), incremental loads, backfills, schema checks, data-quality tests (nulls, duplicates, freshness, row counts), monitoring and lineage.
- **File formats**: CSV/JSON (row text, slow), **Parquet/ORC** (columnar, compressed, fast scans, predicate pushdown), Avro (row, schema evolution, streaming). Partition by date; avoid tiny files.
- **Modelling**: star schema = fact table (events, measures) + dimension tables (who, what, when); **SCD type 2** keeps history of changing dimensions; normalised for OLTP, denormalised for analytics.
- **CDC** (change data capture) streams database changes; **data contracts** define schemas between teams.

PYSPARK

- Distributed processing: data split into **partitions** across executors; the driver plans work. **Lazy evaluation**: transformations (select, filter, join, groupBy) build a plan; **actions** (count, show, collect, write) run it.
- **Narrow** transformations stay within a partition; **wide** ones (join, groupBy, distinct, orderBy) cause a **shuffle** (network + disk): the main cost.

```
from pyspark.sql import SparkSession, functions as F
spark = SparkSession.builder.getOrCreate()
df = spark.read.parquet("s3://b/events/")
out = (df.filter(F.col("dt") >= "2025-01-01")
      .groupBy("user_id")
      .agg(F.count("*").alias("n"),
           F.sum("amt").alias("rev"))
      .join(F.broadcast(users), "user_id", "left"))
out.write.mode("overwrite").parquet("o/")
```

- **Speed**: filter and select early, **broadcast** the small side of a join, avoid wide ops you do not need, use built-in functions (not Python UDFs), cache reused data (df.cache()), repartition sensibly (coalesce to reduce), fix **skew** (one huge key: salting, broadcast, AQE).
- **collect()** pulls all data to the driver: can crash. DataFrames are optimised by Catalyst; RDDs are lower level.
- **Window**:

```
F.row_number().over(Window.partitionBy("g").orderBy("d"))
```

DATABASES

Type	Examples	Good for
Relational (SQL)	Postgres, MySQL	transactions, joins, ACID
Key-value	Redis, DynamoDB	fast lookups, caching, sessions
Document	MongoDB	flexible JSON
Wide-column	Cassandra, HBase	huge write volume
Graph	Neo4j	relationships
Columnar warehouse	BigQuery, Snowflake	analytic scans
Vector	pgvector, Pinecone, FAISS	embedding search

- **OLTP** many small transactions (row store); **OLAP** big aggregations (column store).
- **CAP**: under a network partition a distributed system must choose consistency or availability. **ACID** vs **BASE** (eventual consistency). Sharding splits data across machines; replication copies it for reliability.
- Choose by access pattern: transactions → relational; scans → warehouse; fast lookup → key-value; files → object storage.

CLOUD SERVICE MAP

- Format: need = **AWS / GCP / Azure**.
- Object storage = S3 / Cloud Storage / Blob Storage.
- Warehouse = Redshift / BigQuery / Synapse.
- Managed Spark = EMR, Glue / Dataproc / Databricks.
- ML platform = SageMaker / Vertex AI / Azure ML.
- Serverless functions = Lambda / Cloud Functions / Functions.
- Streaming = Kinesis, MSK / Pub/Sub / Event Hubs.
- Orchestration = MWAA, Step Functions / Composer / Data Factory.
- IAM = identities and permissions (least privilege). Cost levers: right-size, spot instances, partition and prune, storage tiers, turn clusters off.

GIT

```
git clone URL; git status; git diff
git add -p; git commit -m "msg"
git switch -c feature/x
git push -u origin feature/x
git pull --rebase; git fetch
git merge main; git rebase main
git log --oneline --graph; git blame f
git stash; git stash pop
git restore f; git reset --soft HEAD~1
git revert SHA # safe undo on
shared branch
git cherry-pick SHA; git tag v1.0
```

- Branch → small commits → push → pull request → review → merge. **Merge** keeps history with a merge commit; **rebase** replays your commits for a linear history: never rebase shared branches.
- Conflict: edit markers <<<<<<, pick the right content, git add, continue.
.gitignore for data, secrets, .venv. A committed secret is leaked: **rotate it**, then remove it.

LINUX AND SHELL

```
ls -lah; cd; pwd; mkdir -p d; rm -r d
head -n 5 f; tail -f log; less f; wc -l f
grep -rn "err" . | sort | uniq -c | sort -nr
cut -d, -f1,3 f.csv; awk -F, '{s+=$2} END{print s}' f
find . -name "*.py"; du -sh *; df -h
chmod +x run.sh; ps aux | grep py; kill -9 PID
cat a.txt | tr a-z A-Z > b.txt; command >> log 2>&1
ssh user@host; scp f host:/path; curl -O URL
export VAR=1; env; which python; cron: 0 2 * * *
```

Pipes | chain commands; > overwrites, >> appends, 2>&1 merges errors.

ENVIRONMENTS AND DOCKER

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt; pip freeze > requirements.txt
conda create -n x python=3.11; conda activate x
docker build -t app .; docker run -p 8000:8000 app
docker ps; docker logs -f ID; docker exec -it ID sh
docker compose up -d
```

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0"]
```

- Image** = blueprint; **container** = running instance. Copy requirements first to use layer caching. Same image runs everywhere: reproducibility. Keep secrets out of images (env vars/secret managers). Pin versions, set random seeds.

BI TOOLS AND DASHBOARDS

- Tableau / Power BI / Looker** connect to data, build charts, filters, calculated fields, and dashboards. Know: measures vs dimensions, extracts vs live connections, LOD expressions/DAX basics.
- A good dashboard answers specific questions: key metric first, trend, breakdown, filters, consistent definitions, last-refreshed date, few colours.
- Ask who decides what from it; agree metric definitions; avoid vanity metrics.

EXCEL ESSENTIALS

- Lookups: XLOOKUP(key, keys, vals) (or VLOOKUP(key, table, col, FALSE); INDEX/MATCH). #N/A = key missing, type mismatch or stray space (TRIM).
- Aggregation: SUMIFS, COUNTIFS, AVERAGEIFS; logic IF, IFS, IFERROR; text LEFT/MID/RIGHT, TEXTJOIN, SPLIT via Text-to-columns; dates EOMONTH, DATEDIF, TEXT(d, "yyyy-mm").
- Pivot tables** = GROUP BY (rows = groups, values = aggregates, columns = pivot); slicers filter; refresh after data changes. Lookup = left join.
- Tips: absolute references \$A\$1, named ranges, Ctrl+Shift+arrows, data validation, conditional formatting, Power Query for repeatable cleaning.

PIPELINES IN PRACTICE

- Incremental load / upsert:** MERGE INTO tgt USING src ON key WHEN MATCHED THEN UPDATE ... WHEN NOT MATCHED THEN INSERT ...; keep a **watermark** (last processed timestamp); make reruns safe with dedupe on a key.

- Streaming:** event time vs processing time; **windows** (tumbling, sliding, session); **watermarks** for late data; delivery semantics at-most-once, at-least-once, **exactly-once** (idempotent writes + transactions). Kafka: topics, partitions, consumer groups, offsets.
- Data quality checks:** not-null, unique keys, accepted values, referential integrity, freshness, row-count and distribution drift (Great Expectations, dbt tests).

```
SELECT COUNT(*) - COUNT(DISTINCT id) AS
dups,
SUM(CASE WHEN amt IS NULL THEN 1
ELSE 0 END) AS nulls,
MAX(loaded_at) AS freshness
FROM fact_orders;
```

- APIs and JSON:** REST verbs GET/POST/PUT/DELETE, status codes (200 ok, 400 client error, 401/403 auth, 404, 429 rate limit, 500 server error); requests.get(url, timeout=10).json(); paginate and retry with backoff.
- CI/CD:** run tests and linters on every push (GitHub Actions); build the image; deploy automatically to staging, then production.
- Testing:** unit tests for transformations, sample-data integration tests, schema tests.

COST AND PERFORMANCE HABITS

- Store as Parquet, partition by date, prune partitions in queries, avoid SELECT * on columnar warehouses (pay per bytes scanned).
- Right-size clusters, auto-terminate idle ones, use spot capacity for batch, cache hot results.
- Monitor job duration, failures, data freshness, and cost per run.

SAY IT IN 20 SECONDS

- "ELT loads raw first and transforms in the warehouse."
- "Spark is lazy; shuffles are the cost; broadcast the small table."
- "Choose the database by access pattern: transactions, scans, lookups."
- "Merge keeps history, rebase rewrites it; never rebase shared branches."
- "Docker bundles code and environment so it runs the same everywhere."